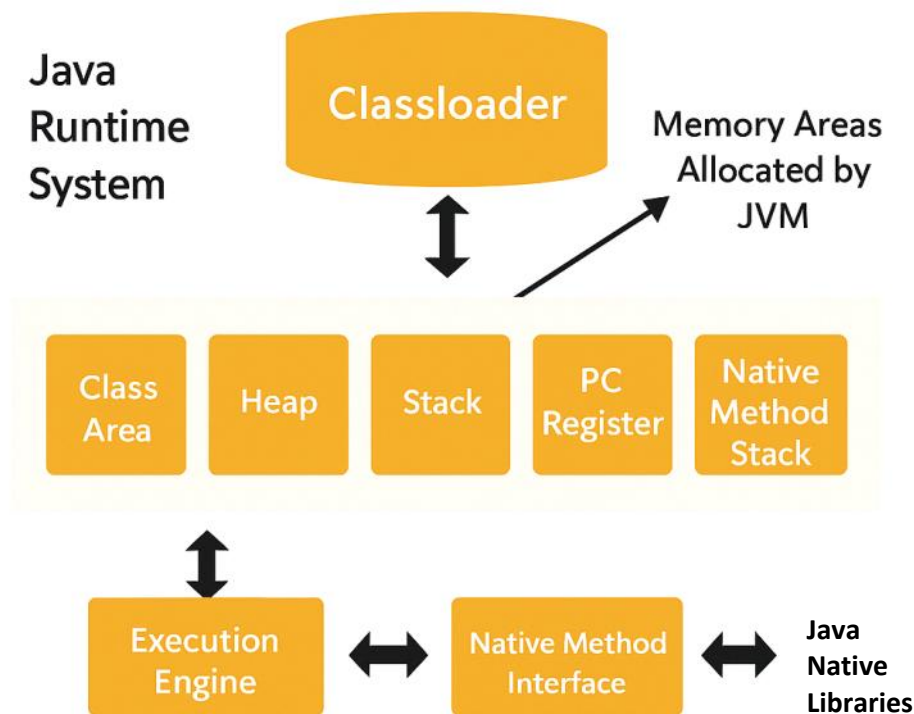


How JVM Works Internally

JVM Architecture



ClassLoader: The First Step

- When you run a Java program, ClassLoader is the first component to step in.
- It takes the .class files (compiled bytecode) and loads them into memory.
- Loads classes **lazily** – only when needed.
- Keeps memory usage optimized.

Runtime Data Areas

- After class loading, bytecode is handed over to Runtime Data Areas.
- This area is like the **brain of the JVM**.
- Holds everything required to run the program. Includes:
 - Class Area
 - Heap
 - Stack
 - Program Counter Register
 - Native Method Stack

Execution Engine

- Responsible for actually executing bytecode
- Doesn't interpret every line every time
- Uses **Just-In-Time compilation (JIT)** for performance

How JIT Works

- First time a method runs → interpreted line-by-line
- If called repeatedly → JIT compiles it to native machine code
- Makes Java perform nearly as fast as C++

Native Method Interface

- Acts as a bridge between the JVM and external libraries
- Let Java call C functions or talk to the OS directly
- Enables system-level interaction from Java